# Documentación de proyecto.

Construcción Matemática

## Documentación para equipo

**Nombre:** Luis Joshua Rodriguez Urbano
**Boleta:** 2026600631
**Profesor(a):** Juan Manuel Figueroa Flores
**Fecha:** 16 de octubre de 2026

## 1. Introducción

## 2. Tablas Entidades (Maestras)

### 2.1. Tabla: usuarios

```sql
CREATE TABLE IF NOT EXISTS usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_usuario`: Identificador único para cada cuenta registrada en el sistema.
- `username`: Nombre visible y único para permitir el inicio de sesión del usuario.
- `password_hash`: Almacenamiento de la contraseña encriptada por seguridad.
- `created_at`: Registro automático del momento exacto en que se creó la cuenta.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| usuarios | rel_user_playlist |
| id_usuario | id_usuario |

### 2.2. Tabla: artistas

```sql
CREATE TABLE IF NOT EXISTS artistas (
    id_artista INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_artista`: Identificador único de cada músico, intérprete o banda.
- `nombre`: Nombre artístico o comercial bajo el que se publican las obras musicales.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| artistas<br>id_artista | rel_art_album<br>id_artista |
| artistas<br>id_artista | rel_art_track<br>id_artista |

### 2.3. Tabla: albumes

```sql
CREATE TABLE IF NOT EXISTS albumes (
    id_album INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    descripcion TEXT NULL,
    anio_lanzamiento YEAR NULL,
    portada_ruta VARCHAR(255) NULL
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_album`: Identificador único para cada disco o recopilación.
- `titulo`: Nombre comercial del álbum.
- `descripcion`: Espacio para información editorial o contexto del trabajo discográfico.
- `anio_lanzamiento`: Año en el que la obra fue publicada oficialmente.
- `portada_ruta`: Ubicación en el disco duro o URL para cargar la imagen de la carátula.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| albumes<br>id_album | rel_tracks_album<br>id_album |
| albumes<br>id_album | rel_art_album<br>id_album |
| albumes<br>id_album | rel_notes_album<br>id_album |

### 2.4. Tabla: canciones

```sql
CREATE TABLE IF NOT EXISTS canciones (
    id_cancion INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    duracion_segundos INT NOT NULL DEFAULT 0,
    ruta_archivo VARCHAR(255) NULL
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_cancion`: Identificador único para cada pista de audio.
- `titulo`: Nombre de la obra musical.
- `duracion_segundos`: Tiempo total de reproducción para mostrar en la interfaz.
- `ruta_archivo`: Ruta exacta en el sistema de archivos para que Python pueda reproducir el MP3/FLAC.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| canciones<br>id_cancion | rel_tracks_album<br>id_cancion |
| canciones<br>id_cancion | rel_tracks_playlist<br>id_cancion |
| canciones<br>id_cancion | rel_art_track<br>id_cancion |

### 2.5. Tabla: playlists

```
CREATE TABLE IF NOT EXISTS playlists (
    id_playlist INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_playlist`: Identificador único de la lista de reproducción.
- `nombre`: Título personalizado asignado a la colección.
- `descripcion`: Resumen o temática de la lista creada por el usuario.
- `created_at`: Registro de la fecha y hora de creación de la lista.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| playlists<br>id_playlist | rel_tracks_playlist<br>id_playlist |
| playlists<br>id_playlist | rel_user_playlist<br>id_playlist |

### 2.6. Tabla: notas

```
CREATE TABLE IF NOT EXISTS notas (
    id_nota INT AUTO_INCREMENT PRIMARY KEY,
    contenido TEXT NOT NULL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_nota`: Identificador único para el comentario o apunte.
- `contenido`: El texto íntegro con el análisis, reseña o impresión personal.
- `fecha`: Momento cronológico exacto en que se guardó la nota.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| notas<br>id_nota | rel_notes_album<br>id_nota |

## 3. Tablas Intermedias (Relaciones)

### 3.1. Tabla: rel_tracks_album

```sql
CREATE TABLE IF NOT EXISTS rel_tracks_album (
    id_album INT NOT NULL,
    id_cancion INT NOT NULL,
    numero_pista INT NULL,
    PRIMARY KEY (id_album, id_cancion)
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_album`: Referencia lógica al álbum contenedor.
- `id_cancion`: Referencia lógica a la canción incluida en el disco.
- `numero_pista`: Orden numérico que ocupa la pista dentro del álbum.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| rel_tracks_album<br>id_album | albumes<br>id_album |
| rel_tracks_album<br>id_cancion | canciones<br>id_cancion |

### 3.2. Tabla: rel_tracks_playlist

```sql
CREATE TABLE IF NOT EXISTS rel_tracks_playlist (
    id_playlist INT NOT NULL,
    id_cancion INT NOT NULL,
    posicion INT NULL,
    agregado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_playlist, id_cancion)
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_playlist`: Referencia a la lista de reproducción receptora.
- `id_cancion`: Referencia a la canción añadida.
- `posicion`: Turno de reproducción que tiene la canción dentro de la lista para mantener un orden.
- `agregado_en`: Auditoría del instante en que el usuario añadió la canción.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| rel_tracks_playlist<br>id_playlist | playlists<br>id_playlist |
| rel_tracks_playlist<br>id_cancion | canciones<br>id_cancion |

### 3.3. Tabla: rel_art_track

```
CREATE TABLE IF NOT EXISTS rel_art_track (
    id_artista INT NOT NULL,
    id_cancion INT NOT NULL,
    rol VARCHAR(50) DEFAULT 'Principal',
    PRIMARY KEY (id_artista, id_cancion)
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_artista`: Referencia al músico participante.
- `id_cancion`: Referencia a la canción interpretada o producida.
- `rol`: Etiqueta para distinguir colaboraciones (vocalista principal, featuring, productor).

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| rel_art_track<br>id_artista | artistas<br>id_artista |
| rel_art_track<br>id_cancion | canciones<br>id_cancion |

### 3.4. Tabla: rel_art_album

```
CREATE TABLE IF NOT EXISTS rel_art_album (
    id_artista INT NOT NULL,
    id_album INT NOT NULL,
    PRIMARY KEY (id_artista, id_album)
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_artista`: Referencia al autor o creador musical.
- `id_album`: Referencia al disco publicado.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| rel_art_album<br>id_artista | artistas<br>id_artista |
| rel_art_album<br>id_album | albumes<br>id_album |

### 3.5. Tabla: rel_notes_album

```
CREATE TABLE IF NOT EXISTS rel_notes_album (
    id_nota INT NOT NULL,
    id_album INT NOT NULL,
    PRIMARY KEY (id_nota, id_album)
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_nota`: Referencia al apunte escrito por el usuario.
- `id_album`: Referencia al disco sobre el cual se hizo el apunte.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| rel_notes_album<br>id_nota | notas<br>id_nota |
| rel_notes_album<br>id_album | albumes<br>id_album |

### 3.6. Tabla: rel_user_playlist

```
CREATE TABLE IF NOT EXISTS rel_user_playlist (
    id_usuario INT NOT NULL,
    id_playlist INT NOT NULL,
    es_propietario BOOLEAN DEFAULT TRUE,
    PRIMARY KEY (id_usuario, id_playlist)
) ENGINE=InnoDB;
```

**¿Por qué se creó cada columna?**

- `id_usuario`: Referencia a la cuenta de usuario del sistema.
- `id_playlist`: Referencia a la lista musical interactuada.
- `es_propietario`: Indicador para diferenciar si el usuario es creador de la playlist o si solo la guardó como favorita.

**Referenciaciones**

| Tabla Origen | Tabla Destino |
|---|---|
| rel_user_playlist<br>id_usuario | usuarios<br>id_usuario |
| rel_user_playlist<br>id_playlist | playlists<br>id_playlist |

## 4. Relaciones (Llaves Foráneas)

En esta sección se definen las restricciones de integridad referencial aplicadas mediante sentencias `ALTER TABLE`, las cuales enlazan las entidades maestras a través de las tablas intermedias. Todas incluyen reglas de eliminación y actualización en cascada (`ON DELETE CASCADE ON UPDATE CASCADE`) para evitar registros huérfanos.

### 4.1. Relación: Álbum - Canciones (`rel_tracks_album`)

```
ALTER TABLE rel_tracks_album
    ADD CONSTRAINT fk_rta_album FOREIGN KEY (id_album) REFERENCES albumes(id_album),
    ADD CONSTRAINT fk_rta_cancion FOREIGN KEY (id_cancion) REFERENCES canciones(id_cancion
        );
```

- **Columnas relacionadas:** La columna `id_album` se vincula con `albumes(id_album)` y la columna `id_cancion` se vincula con `canciones(id_cancion)`.
- **Tipo de relación:** Muchos a Muchos (N:M) entre `albumes` y `canciones`.
- **¿Por qué se relaciona?** Para estructurar el contenido de un disco. Un álbum contiene múltiples canciones, y una misma pista de audio puede pertenecer a más de un álbum (por ejemplo, álbumes de estudio originales frente a discos de grandes éxitos o recopilaciones) sin necesidad de duplicar el archivo físico en el sistema.

### 4.2. Relación: Playlist - Canciones (`rel_tracks_playlist`)

```
ALTER TABLE rel_tracks_playlist
    ADD CONSTRAINT fk_rtp_playlist FOREIGN KEY (id_playlist) REFERENCES playlists(
        id_playlist),
    ADD CONSTRAINT fk_rtp_cancion FOREIGN KEY (id_cancion) REFERENCES canciones(id_cancion
        );
```

- **Columnas relacionadas:** La columna `id_playlist` se vincula con `playlists(id_playlist)` y la columna `id_cancion` se vincula con `canciones(id_cancion)`.
- **Tipo de relación:** Muchos a Muchos (N:M) entre `playlists` y `canciones`.
- **¿Por qué se relaciona?** Para permitir al usuario crear agrupaciones personalizadas. Una lista de reproducción está compuesta por múltiples canciones, y una misma canción puede ser agregada a docenas de playlists diferentes.

### 4.3. Relación: Artista - Canción (`rel_art_track`)

```
ALTER TABLE rel_art_track
    ADD CONSTRAINT fk_rat_artista FOREIGN KEY (id_artista) REFERENCES artistas(id_artista)
        ,
    ADD CONSTRAINT fk_rat_cancion FOREIGN KEY (id_cancion) REFERENCES canciones(id_cancion
        );
```

- **Columnas relacionadas:** La columna `id_artista` se vincula con `artistas(id_artista)` y la columna `id_cancion` se vincula con `canciones(id_cancion)`.
- **Tipo de relación:** Muchos a Muchos (N:M) entre `artistas` y `canciones`.
- **¿Por qué se relaciona?** Para gestionar correctamente los créditos musicales. Un artista interpreta o produce muchas canciones, pero a su vez, una sola canción puede tener la participación de múltiples artistas simultáneamente (colaboraciones, artistas invitados o *featurings*).

### 4.4. Relación: Artista - Álbum (`rel_art_album`)

```
ALTER TABLE rel_art_album
    ADD CONSTRAINT fk_raa_artista FOREIGN KEY (id_artista) REFERENCES artistas(id_artista)
        ,
    ADD CONSTRAINT fk_raa_album FOREIGN KEY (id_album) REFERENCES albumes(id_album);
```

- **Columnas relacionadas:** La columna `id_artista` se vincula con `artistas(id_artista)` y la columna `id_album` se vincula con `albumes(id_album)`.
- **Tipo de relación:** Muchos a Muchos (N:M) entre `artistas` y `albumes`.
- **¿Por qué se relaciona?** Para atribuir la autoría a nivel de disco completo. Si bien lo normal es que un álbum sea de un solo artista (1:N), existen álbumes colaborativos creados en conjunto por dos o más artistas principales, lo que hace necesaria una estructura de muchos a muchos.

### 4.5. Relación: Notas - Álbum (`rel_notes_album`)

```
ALTER TABLE rel_notes_album
    ADD CONSTRAINT fk_rna_nota FOREIGN KEY (id_nota) REFERENCES notas(id_nota),
    ADD CONSTRAINT fk_rna_album FOREIGN KEY (id_album) REFERENCES albumes(id_album);
```

- **Columnas relacionadas:** La columna `id_nota` se vincula con `notas(id_nota)` y la columna `id_album` se vincula con `albumes(id_album)`.
- **Tipo de relación:** Muchos a Muchos (N:M) a nivel de esquema lógico.
- **¿Por qué se relaciona?** Para vincular los comentarios, reseñas o apuntes del usuario con un trabajo discográfico específico. Permite que un álbum acumule múltiples notas a lo largo del tiempo, y flexibiliza el modelo si una misma nota (por ejemplo, una reseña general de una década) aplicara a más de un disco.

### 4.6. Relación: Usuario - Playlist (`rel_user_playlist`)

```
ALTER TABLE rel_user_playlist
    ADD CONSTRAINT fk_rup_usuario FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
        ,
    ADD CONSTRAINT fk_rup_playlist FOREIGN KEY (id_playlist) REFERENCES playlists(
        id_playlist);
```

- **Columnas relacionadas:** La columna `id_usuario` se vincula con `usuarios(id_usuario)` y la columna `id_playlist` se vincula con `playlists(id_playlist)`.
- **Tipo de relación:** Muchos a Muchos (N:M) entre `usuarios` y `playlists`.
- **¿Por qué se relaciona?** Para administrar la propiedad e interacción social del sistema. Un usuario puede tener muchas playlists, y una playlist creada por alguien podría ser guardada como "favorita" por múltiples usuarios diferentes, operando de manera similar al modelo de Spotify.

## 5. Documentación del Sistema: Archivo Principal (`main.py`)

El archivo `main.py` funciona como el controlador principal de la aplicación. Gestiona la interfaz de línea de comandos (CLI), el flujo de pantallas y la interacción directa con el usuario mediante menús de opciones.

### 5.1. Importación de Módulos y Funciones

```
import os
import time

# Importamos las funciones necesarias
from funciones import (add_artist, add_user, add_album, add_track, add_playlist,
                       link_track_to_album, verify_user, show_albums, show_artists,
                       show_tracks, show_playlists)
```

**¿Para qué sirve esta sección?**

- `os`: Librería nativa de Python utilizada para ejecutar comandos del sistema operativo (necesaria para limpiar la terminal).
- `time`: Se emplea para generar pequeñas pausas visuales (con `time.sleep()`) y permitir que el usuario lea los mensajes antes de que la pantalla se actualice.
- `from funciones import ...`: Importa la lógica de negocio (operaciones de inserción, consulta y validación en la base de datos) desde un archivo modular externo llamado `funciones.py`, manteniendo el código principal limpio.

### 5.2. Funciones de Interfaz Gráfica (CLI)

```
def limpiar_pantalla():
    os.system('cls' if os.name == 'nt' else 'clear')

def mostrar_encabezado():
    print("=" * 50)
    print("ADMINISTRADOR DE MSICA LOCAL".center(50))
    print("=" * 50)
```

**¿Para qué sirve cada función?**

- `limpiar_pantalla()`: Detecta el sistema operativo subyacente. Si es Windows (`'nt'`), ejecuta el comando `cls`; si es macOS o Linux, ejecuta `clear`. Esto mantiene la consola ordenada borrando el historial de texto anterior.
- `mostrar_encabezado()`: Imprime un título centrado y enmarcado con caracteres `=` para darle una apariencia estética y formal a la aplicación.

### 5.3. Menú Principal (Sesión Iniciada)

```
def menu_principal(id_usuario_actual, nombre_usuario):
    """Este es el menu al que solo accedes SI iniciaste sesion"""
    while True:
        limpiar_pantalla()
```

```
        mostrar_encabezado()
        print(f"Bienvenido/a, {nombre_usuario.upper()} (Tu ID: {id_usuario_actual})\n")
        # ... (Impresion de opciones 1 al 9 y 0) ...

        opcion = input("Elige una opcion (0-9): ")

        if opcion == '1':
            # Logica para agregar artista
        elif opcion == '2':
            # Logica para agregar album con campos opcionales
        # ... (Resto de condicionales) ...
        elif opcion == '0':
            print("Cerrando sesion...")
            time.sleep(1)
            break
```

**¿Para qué sirve esta función?**

- **Gestión de estado:** Recibe el `id_usuario_actual` y el `nombre_usuario` para personalizar la experiencia y saber quién está ejecutando las acciones (vital para crear playlists asociadas a ese usuario).
- **Bucle infinito (`while True`):** Mantiene al usuario dentro del sistema realizando múltiples operaciones hasta que decide explícitamente cerrar sesión (opción `0`), lo que rompe el bucle con `break`.
- **Enrutamiento de opciones:** Utiliza un bloque `if-elif` para capturar la decisión del usuario, solicitar los datos pertinentes (usando `input`) y llamar a la función correspondiente importada desde `funciones.py`.
- **Manejo de nulos:** Contiene lógica básica para transformar inputs vacíos en valores nulos (e.g., `desc if desc != else None`) antes de enviarlos a la base de datos.

### 5.4. Punto de Entrada y Pantalla de Bloqueo (Login)

```
def main():
    """Esta es la pantalla de bloqueo (Login/Registro)"""
    while True:
        # ... (Impresion de opciones de inicio o registro) ...
        opcion = input("Elige una opcion: ")

        if opcion == '1':
            # Iniciar sesion
            user = input("Usuario: ")
            password = input("Contrasena: ")
            id_logueado = verify_user(user, password)

            if id_logueado:
                menu_principal(id_logueado, user)
            else:
                print("Usuario o contrasena incorrectos.")

        elif opcion == '2':
            # Registro de nuevo usuario
            add_user(user, password)
```

```
21
22          elif opcion == '0':
23              break
24
25  if __name__ == '__main__':
26      main()
```

**¿Para qué sirve esta sección final?**

- `def main()`: Actúa como el primer filtro de seguridad de la aplicación. Obliga al usuario a autenticarse (`verify_user`) o a crear una cuenta nueva (`add_user`) antes de poder interactuar con la base de datos musical.
- **Transición de estado:** Si la función `verify_user` retorna un ID válido, el programa transfiere el control a la función `menu_principal()`, ïniciando sesión.[ef]ectivamente.
- `if __name__ == '__main__':`: Es un patrón estándar de Python que asegura que la función `main()` solo se ejecute si el archivo se corre directamente (ej. `python main.py`), y no si es importado como módulo en otro script.

## 6. Documentación del Sistema: Lógica de Negocio (`funciones.py`)

El archivo `funciones.py` concentra toda la lógica de acceso a datos (CRUD). Actúa como un puente entre la interfaz de usuario gestionada en `main.py` y la base de datos MySQL. Esta separación de responsabilidades asegura que el código de la consola no contenga consultas SQL directas, haciendo el sistema más modular y escalable.

### 6.1. Autenticación y Gestión de Usuarios

```python
def add_user(username, password_hash):
    # Inserta un nuevo usuario en la base de datos
    # ...

def verify_user(username, password_hash):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "SELECT id_usuario FROM usuarios WHERE username = %s AND password_hash =
                %s"
            cursor.execute(sql, (username, password_hash))
            resultado = cursor.fetchone()

            if resultado:
                return resultado[0]
            return None
        # ...
```

**Relación con el sistema y funcionamiento:**

- **Seguridad y Acceso:** La función `verify_user` es invocada directamente por la pantalla de bloqueo en `main.py`. Si las credenciales coinciden, retorna el `id_usuario`, el cual `main.py` guarda en memoria como la "sesión activa" (`id_logueado`).
- **Prevención de Inyección SQL:** Los valores no se concatenan directamente en la cadena SQL, sino que se pasan como tupla `(username, password_hash)` al método `cursor.execute()`, delegando el saneamiento de datos a la librería de conexión.

### 6.2. Inserción de Entidades Maestras y Recuperación de IDs

```python
def add_album(titulo, descripcion=None, anio_lanzamiento=None, portada_ruta=None):
    # ... (Conexion a BD) ...
            sql = "INSERT INTO albumes (titulo, descripcion, anio_lanzamiento, portada_ruta
                ) VALUES (%s, %s, %s, %s)"
            cursor.execute(sql, (titulo, descripcion, anio_lanzamiento, portada_ruta))
            cnx.commit()

            id_creado = cursor.lastrowid
            print(f"Album '{titulo}' agregado con el ID: {id_creado}.")
            return id_creado
    # ...
```

**¿Para qué sirve esta técnica?**

- Las funciones `add_album` y `add_track` utilizan la propiedad `cursor.lastrowid`. Esto permite capturar inmediatamente el ID autoincremental que MySQL acaba de generar.
- Es de suma utilidad para el usuario de `main.py`, ya que al momento de insertar un disco o canción, el sistema le imprime en pantalla el ID exacto que necesitará utilizar en el paso posterior cuando llame a `link_track_to_album` (opción 4 del menú principal).

### 6.3. Transacciones Complejas: Creación de Playlists

```
def add_playlist(nombre, id_usuario, descripcion=None):
    # ... (Conexion a BD) ...
        try:
            cursor = cnx.cursor()
            # 1. Crear la playlist
            sql_playlist = "INSERT INTO playlists (nombre, descripcion) VALUES (%s, %s)"
            cursor.execute(sql_playlist, (nombre, descripcion))
            id_playlist = cursor.lastrowid

            # 2. Vincularla al usuario como propietario
            sql_relacion = "INSERT INTO rel_user_playlist (id_usuario, id_playlist,
                es_propietario) VALUES (%s, %s, True)"
            cursor.execute(sql_relacion, (id_usuario, id_playlist))

            cnx.commit()
        except Exception as e:
            cnx.rollback()
            print(f"Error al crear la playlist. Se deshicieron los cambios: {e}")
    # ...
```

**Conexión con el Menú Principal y Control de Errores:**

- **Uso de la Sesión:** Esta función recibe el `id_usuario`, el cual proviene del parámetro `id_usuario_actual` que `main.py` mantiene vivo tras el inicio de sesión.
- **Atomicidad (Transacciones):** Es la única función de inserción que afecta a dos tablas simultáneamente (`playlists` y `rel_user_playlist`). Si el paso 1 tiene éxito pero el paso 2 falla, la instrucción `cnx.rollback()` en el bloque `except` revierte la inserción de la playlist, evitando que queden "playlists huérfanas"sin dueño en el sistema. Si ambos pasos son exitosos, `cnx.commit()` guarda todo definitivamente.

### 6.4. Consultas Relacionales (JOINs) y Formateo de Salida

```
def show_albums():
    # ... (Conexion a BD) ...
            sql = """
                SELECT a.id_album, a.titulo, a.descripcion, a.anio_lanzamiento, c.titulo, r
                    .numero_pista
                FROM albumes a
                LEFT JOIN rel_tracks_album r ON a.id_album = r.id_album
                LEFT JOIN canciones c ON r.id_cancion = c.id_cancion
                ORDER BY a.id_album, r.numero_pista
            """
```

```
            cursor.execute(sql)
            resultados = cursor.fetchall()

            if resultados:
                album_actual = None
                for fila in resultados:
                    id_album, titulo_album, desc, anio, titulo_cancion, num_pista = fila

                    if id_album != album_actual:
                        # Imprimir cabecera del album solo una vez
                        print(f"[*] {titulo_album.upper()} (ID: {id_album})")
                        album_actual = id_album

                    if titulo_cancion:
                        # Imprimir las canciones tabuladas debajo del album
                        pista = num_pista if num_pista else "-"
                        print(f" -> {pista}. {titulo_cancion}")
    # ...
```

**¿Por qué es importante esta consulta?**

- **Consulta Estructurada con JOIN:** A diferencia de listar solo los álbumes crudos, utiliza `LEFT JOIN` para unir `albumes`, la tabla puente `rel_tracks_album` y la tabla `canciones`. Esto permite traer toda la información jerárquica en un solo viaje a la base de datos.
- **Lógica de Presentación en Python:** Para evitar imprimir los datos del álbum repetidas veces (una por cada canción), la variable `album_actual` funciona como un çorte de control". Solo imprime la cabecera del álbum cuando detecta un `id_album` nuevo, agrupando visualmente las canciones debajo de su respectivo disco para mejorar la experiencia del usuario final en la terminal.

## 7. Documentación del Sistema: Capa de Conexión (conexion.py)

El archivo `conexion.py` representa la capa más baja de la arquitectura de la aplicación (Data Connection Layer). Su única responsabilidad es establecer y verificar la comunicación física entre el código de Python y el servidor de la base de datos MySQL, proveyendo un punto centralizado para gestionar las credenciales y el estado del servidor.

### 7.1. Establecimiento de la Conexión y Desglose de Parámetros

```python
import mysql.connector
from mysql.connector import Error

def connect_to_database():
    try:
        # 1. INTENTO DE CONEXION
        cnx = mysql.connector.connect(
            host="localhost",
            user="root",
            password="joshwbe", # Tu contrasena real
            database="music_manager",
            port=3306
        )

        if cnx.is_connected():
            print("Conexion exitosa a la base de datos music_manager!")
            return cnx # Esta es la instruccion clave para entregar la conexion viva
```

**¿Para qué sirve y cómo se estructura?**

- **Desacoplamiento (Modularidad):** `main.py` nunca importa a `conexion.py`. La aplicación sigue un flujo limpio: `main.py` interactúa con `funciones.py`, y solo `funciones.py` importa e invoca a `connect_to_database()`. Centralizar esta lógica facilita enormemente su explicación sistemática, algo ideal al documentar la arquitectura en reportes o al exponer el funcionamiento en presentaciones académicas.
- **Configuración Local y Gestores Relacionales:** La definición explícita de `host="localhost"` y el puerto estándar `3306` está optimizada para entornos de desarrollo con herramientas de gestión (como MySQL Workbench o la terminal). Esto asegura que el script apunte exactamente al motor de base de datos local (usando motores de almacenamiento como InnoDB) donde residen las tablas y esquemas relacionales creados.
- **Retorno del Objeto `cnx`:** La instrucción `return cnx` es el corazón del sistema. Entrega el objeto de conexión "vivo.ª `funciones.py`, permitiendo que las consultas que resuelven las relaciones entre múltiples tablas (JOINs, inserciones, etc.) puedan generar sus cursores (`cnx.cursor()`).
- **Nota sobre Buenas Prácticas:** Aunque para fines de este proyecto la contraseña está incrustada directamente (*hardcodeada*) en el código, aislar estas credenciales en un único archivo es un diseño excelente; en versiones futuras o en entornos de producción, esto permite leer las credenciales desde variables de entorno sin modificar el resto de la aplicación.

### 7.2. Manejo Específico de Excepciones y Tolerancia a Fallos

```
    except Error as error:
        # 2. CAPTURAMOS EL ERROR SI OCURRE
        print(f"Error al conectar con MySQL: {error}")
        return None
```

**Relación vital con la estabilidad de `main.py` y `funciones.py`:**

- **Clase `Error` Especializada:** Al importar explícitamente la clase `Error` desde la librería `mysql.connector`, el bloque `except` no captura fallos genéricos de Python, sino excepciones exclusivas del motor de base de datos (por ejemplo, credenciales incorrectas, puerto bloqueado o servidor apagado). Esto proporciona mensajes de diagnóstico mucho más precisos para la resolución de problemas.
- **Retorno de `None` y Prevención de Caídas:** Si la conexión falla, se retorna `None` en lugar de que el programa colapse abruptamente. Al evitar un cierre fatal (*crash*), todas las funciones en `funciones.py` pueden hacer la validación preventiva (`if cnx:`) antes de intentar ejecutar una consulta.
- **Continuidad del Menú Principal:** Gracias a esta robusta gestión de excepciones, si la base de datos se desconecta temporalmente, el bucle `while True` de `main.py` no se rompe. El usuario simplemente verá el mensaje del error de conexión en la terminal, pero será devuelto de manera segura al menú principal, manteniendo una experiencia fluida en la consola.